

✓ HEALTHCARE CLAIMS ANALYSIS

Dataset Information:

- **Each column provides specific information about the patient, their admission, and the healthcare services provided, making this dataset suitable for various data analysis and modeling tasks in the healthcare domain. Here's a brief explanation of each column in the dataset.**
- **Name:** This column represents the name of the patient associated with the healthcare record.
- **Age:** The age of the patient at the time of admission, expressed in years.
- **Gender:** Indicates the gender of the patient, either "Male" or "Female."
- **Blood Type:** The patient's blood type, which can be one of the common blood types (e.g., "A+", "O-", etc.).
- **Medical Condition:** This column specifies the primary medical condition or diagnosis associated with the patient, such as "Diabetes," "Hypertension," "Asthma," and more.
- **Date of Admission:** The date on which the patient was admitted to the healthcare facility.
- **Doctor:** The name of the doctor responsible for the patient's care during their admission.
- **Hospital:** Identifies the healthcare facility or hospital where the patient was admitted.
- **Insurance Provider:** This column indicates the patient's insurance provider, which can be one of several options, including "Aetna," "Blue Cross," "Cigna," "UnitedHealthcare," and "Medicare."
- **Billing Amount:** The amount of money billed for the patient's healthcare services during their admission. This is expressed as a floating-point number.
- **Room Number:** The room number where the patient was accommodated during their admission.
- **Admission Type:** Specifies the type of admission, which can be "Emergency," "Elective," or "Urgent," reflecting the circumstances of the admission.
- **Discharge Date:** The date on which the patient was discharged from the healthcare facility, based on the admission date and a random number of days within a realistic range.
- **Medication:** Identifies a medication prescribed or administered to the patient during their admission. Examples include "Aspirin," "Ibuprofen," "Penicillin," "Paracetamol," and "Lipitor."
- **Test Results:** Describes the results of a medical test conducted during the patient's admission. Possible values include "Normal," "Abnormal," or "Inconclusive," indicating the outcome of the test.

```
1 # import libraries
2 import pandas as pd
3 import seaborn as sns
4 import numpy as np
5 import matplotlib.pyplot as plt
```

```
1 df = pd.read_csv("/content/healthcare_dataset.csv")
```

```
1 df.head()
```

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results	
0	Bobby JacksOn	30	Male	B-	Cancer	2024-01-31	Matthew Smith	Sons and Miller	Blue Cross	18856.281306	328	Urgent	2024-02-02	Paracetamol	Normal	
1	LesLie TErRy	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.327287	265	Emergency	2019-08-26	Ibuprofen	Inconclusive	
2	DaNnY sMitH	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.096079	205	Emergency	2022-10-07	Aspirin	Normal	
3	andrEw waTtS	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang,	Medicare	37909.782410	450	Elective	2020-12-18	Ibuprofen	Abnormal	
4	adriENNE bEll	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.317814	458	Urgent	2022-10-09	Penicillin	Abnormal	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
1 df.shape
```

```
(55500, 15)
```

Inspect the Data

```
1 df.tail()
```

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
55495	eLIZABeTH jaCkSon	42	Female	O+	Asthma	2020-08-16	Joshua Jarvis	Jones-Thompson	Blue Cross	2650.714952	417	Elective	2020-09-15	Penicillin	Abnormal
55496	KYLe pEReZ	61	Female	AB-	Obesity	2020-01-23	Taylor Sullivan	Tucker-Moyer	Cigna	31457.797307	316	Elective	2020-02-01	Aspirin	Normal
55497	HEATHer WaNG	38	Female	B+	Hypertension	2020-07-13	Joe Jacobs DVM	and Mahoney Johnson Vasquez,	UnitedHealthcare	27620.764717	347	Urgent	2020-08-10	Ibuprofen	Abnormal
55498	JENniFER JOnES	43	Male	O-	Arthritis	2019-05-25	Kimberly Curry	Jackson Todd and Castro,	Medicare	32451.092358	321	Elective	2019-05-31	Ibuprofen	Abnormal
55499	jAMES GARCiA	53	Female	O+	Arthritis	2024-04-02	Dennis Warren	Henry Sons and	Aetna	4010.134172	448	Urgent	2024-04-29	Ibuprofen	Abnormal

```
1 df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 55500 entries, 0 to 55499
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Name                  55500 non-null  object
1   Age                   55500 non-null  int64
2   Gender                55500 non-null  object
3   Blood Type            55500 non-null  object
4   Medical Condition     55500 non-null  object
5   Date of Admission     55500 non-null  object
6   Doctor                55500 non-null  object
7   Hospital              55500 non-null  object
8   Insurance Provider    55500 non-null  object
9   Billing Amount         55500 non-null  float64
10  Room Number           55500 non-null  int64
11  Admission Type        55500 non-null  object
12  Discharge Date        55500 non-null  object
13  Medication            55500 non-null  object
14  Test Results          55500 non-null  object
dtypes: float64(1), int64(2), object(12)
memory usage: 6.4+ MB
```

```
1 df.columns

Index(['Name', 'Age', 'Gender', 'Blood Type', 'Medical Condition',
       'Date of Admission', 'Doctor', 'Hospital', 'Insurance Provider',
       'Billing Amount', 'Room Number', 'Admission Type', 'Discharge Date',
       'Medication', 'Test Results'],
      dtype='object')
```

```
1 # Check datatypes
2 df.dtypes
```

	0
Name	object
Age	int64
Gender	object
Blood Type	object
Medical Condition	object
Date of Admission	object
Doctor	object
Hospital	object
Insurance Provider	object
Billing Amount	float64
Room Number	int64
Admission Type	object
Discharge Date	object
Medication	object
Test Results	object

dtype: object

```
1 # Check null values
2 df.isna().sum()
```

	0
Name	0
Age	0
Gender	0
Blood Type	0
Medical Condition	0
Date of Admission	0
Doctor	0
Hospital	0
Insurance Provider	0
Billing Amount	0
Room Number	0
Admission Type	0
Discharge Date	0
Medication	0
Test Results	0

dtype: int64

```
1 # Check duplicated values
2 df.duplicated().sum()
```

np.int64(534)

```
1 # Check data statistical summary
2 df.describe()
```

	Age	Billing Amount	Room Number
count	55500.000000	55500.000000	55500.000000
mean	51.539459	25539.316097	301.134829
std	19.602454	14211.454431	115.243069
min	13.000000	-2008.492140	101.000000
25%	35.000000	13241.224652	202.000000
50%	52.000000	25538.069376	302.000000
75%	68.000000	37820.508436	401.000000
max	89.000000	52764.276736	500.000000



▼ Data Cleaning

```
1 ## Drop duplicated values
2 df = df.drop_duplicates()
```

```
1 # Agian check
2 df.duplicated().sum()
```

```
np.int64(0)
```

```
1 df.shape
```

```
(54966, 15)
```

```
1 ## 01. Clean Name Columns
2 df['Name'] = df['Name'].str.strip().str.title()
```

```
1 df.sample(3)
```

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
18483	Jason Johnson	53	Male	B+	Obesity	2020-10-09	Jennifer Russell	Lynch PLC	Blue Cross	39358.841961	149	Elective	2020-10-20	Lipitor	Normal
47746	Mary Burch	77	Female	A-	Diabetes	2019-11-09	Jessica Johnson	Henry-Rodriguez	UnitedHealthcare	23450.709118	477	Elective	2019-12-08	Lipitor	Normal
50651	Ray Mclean	36	Male	AB-	Asthma	2020-05-01	Carlos Brown	Inc Mccann	UnitedHealthcare	11568.814786	355	Emergency	2020-05-16	Penicillin	Inconclusive

```
1 ## 02. 'Billing Amount'
2 df.loc[df['Billing Amount']<0, 'Billing Amount'] = df['Billing Amount'].mean()
```

1 df.describe()

	Age	Billing Amount	Room Number
count	54966.000000	54966.000000	54966.000000
mean	51.535185	25594.536582	301.124404
std	19.605661	14162.191537	115.223143
min	13.000000	9.238787	101.000000
25%	35.000000	13326.001350	202.000000
50%	52.000000	25544.306284	302.000000
75%	68.000000	37819.858159	401.000000
max	89.000000	52764.276736	500.000000

```
1 ## 03. Change 'Date of Admission' and 'Discharge Date' into date data type
2 df['Date of Admission'] = pd.to_datetime(df['Date of Admission'])
```

```
1 df['Discharge Date'] = pd.to_datetime(df['Discharge Date'])
```

```
1 df.sample(2)
```

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results	Days_of_Stay
29926	Donna Savage	66	Male	AB+	Hypertension	2020-04-13	Gary Glover	Clark, Thomas and Rush	Medicare	33170.085052	326	Urgent	2020-04-15	Aspirin	Inconclusive	
33807	Deborah Moore	85	Male	A-	Asthma	2022-09-04	Brian Reed	Villanueva-Hunter	Aetna	42893.500563	333	Urgent	2022-10-03	Ibuprofen	Inconclusive	

1 # check datatype
2 df.dtypes

0

Name

Age

Gender

Blood Type

Medical Condition

Date of Admission

Doctor

Hospital

Insurance Provider

Billing Amount

Room Number

Admission Type

Discharge Date

Medication

Test Results

object

int64

object

object

object

datetime64[ns]

object

object

object

float64

int64

object

datetime64[ns]

object

object

dtype: object

```
1 ## 04. Create column days_of_stay.  
2 df['Days_of_Stay'] = (df['Discharge Date'] - df['Date of Admission']).dt.days
```

```
1 ## Final cleaning check  
2 df.isna().sum()
```


	0
Name	0
Age	0
Gender	0
Blood Type	0
Medical Condition	0
Date of Admission	0
Doctor	0
Hospital	0
Insurance Provider	0
Billing Amount	0
Room Number	0
Admission Type	0
Discharge Date	0
Medication	0
Test Results	0
Days_of_Stay	0

dtype: int64

1 df.shape

(54966, 16)

1 final_df = df.copy()

1 final_df.sample()

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results	Days_of_Stay
20254	Anna Santana	38	Female	O+	Arthritis	2019-08-24	William Smith	Group Russell	Aetna	42378.924176	233	Urgent	2019-09-17	Aspirin	Abnormal	2

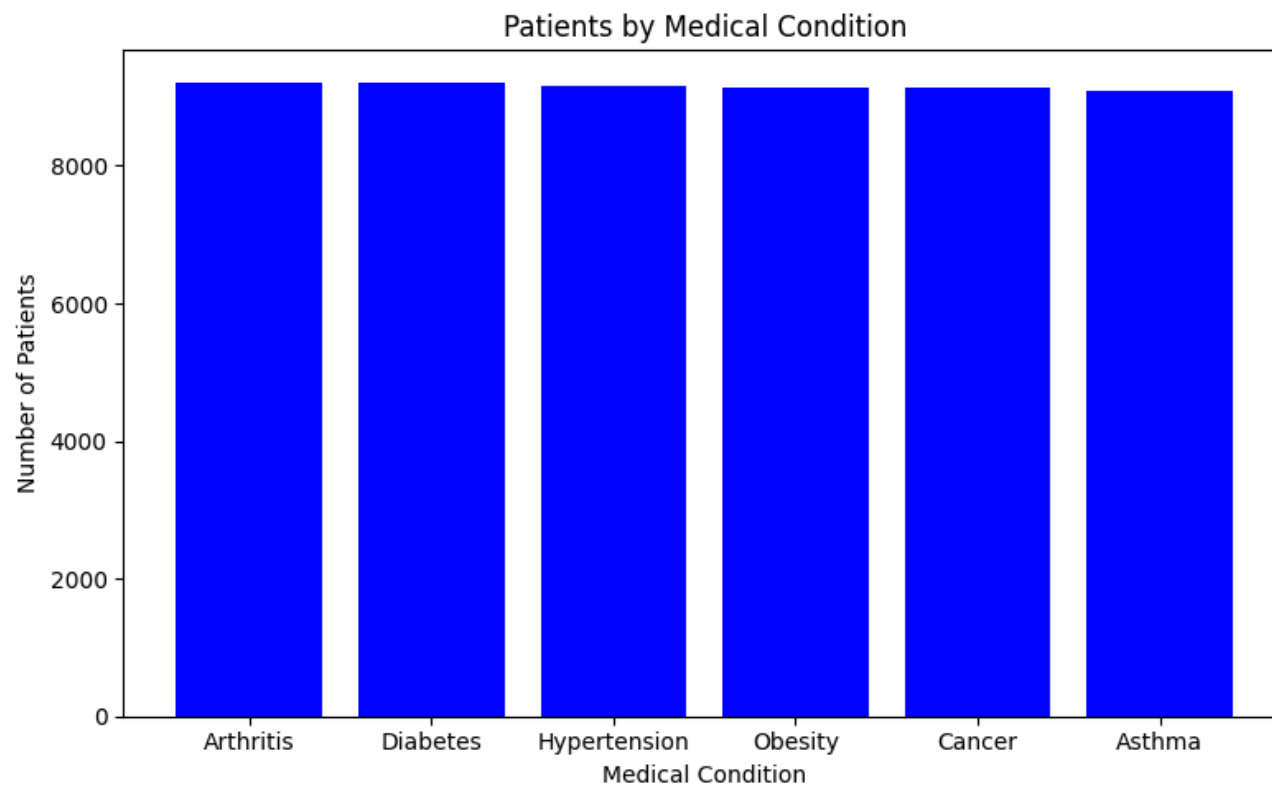
01. Which medical condition affects the most patients across the network?

```
1 condition_count = df['Medical Condition'].value_counts()  
2 condition_count
```

Medical Condition	count
Arthritis	9218
Diabetes	9216
Hypertension	9151
Obesity	9146
Cancer	9140
Asthma	9095

dtype: int64

```
1 import random  
2 plt.figure(figsize=(8,5))  
3 colors = ['b','g','r','c','m','y']  
4 plt.bar(condition_count.index, condition_count.values, color = random.choice(colors))  
5 plt.title('Patients by Medical Condition')  
6 plt.xlabel('Medical Condition')  
7 plt.ylabel('Number of Patients')  
8 # plt.xticks(rotation = 90)  
9 plt.tight_layout()  
10 plt.show()
```

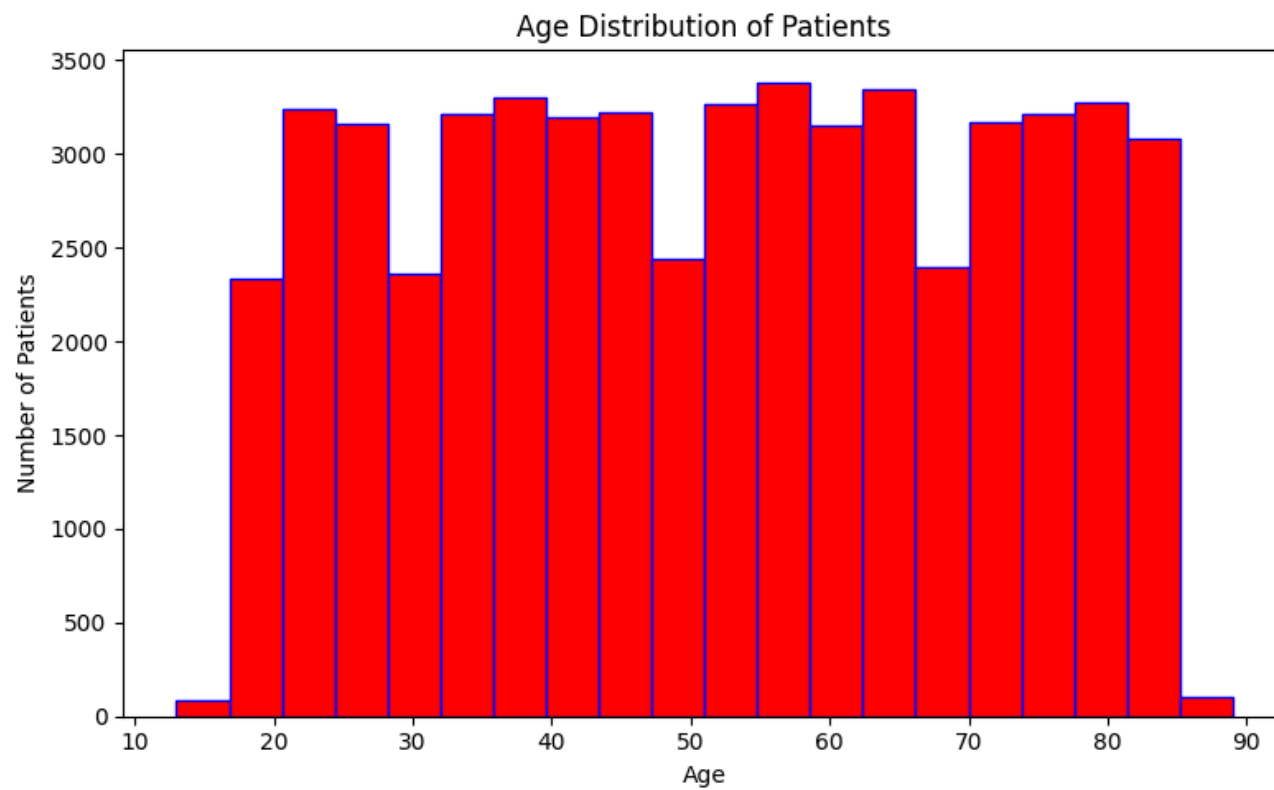


✓ **02. How is patient age distributed across the network?**

```
1 average_age = final_df['Age'].mean()  
2 average_age
```

```
np.float64(51.53518538733035)
```

```
1 plt.figure(figsize=(8,5))  
2 plt.hist(final_df['Age'], bins = 20, color =random.choice(colors), edgecolor = 'b' )  
3 plt.title('Age Distribution of Patients')  
4 plt.xlabel('Age')  
5 plt.ylabel('Number of Patients')  
6 plt.tight_layout()  
7 plt.show()
```



03. How has average treatment cost changed overtime?

```
1 # Create a column Admission_month
2 final_df['Admission_Month'] = final_df['Date of Admission'].dt.to_period('M')
```

```
1 monthly_avg_count = final_df.groupby('Admission_Month')['Billing Amount'].mean()
2 monthly_avg_count
```

Billing Amount	
Admission_Month	
2019-05	25897.449655
2019-06	26324.004064
2019-07	25928.602371
2019-08	25457.102225
2019-09	24915.548929
...	...
2024-01	25836.695757
2024-02	25703.588425
2024-03	25310.583403
2024-04	24930.033214
2024-05	25464.902913

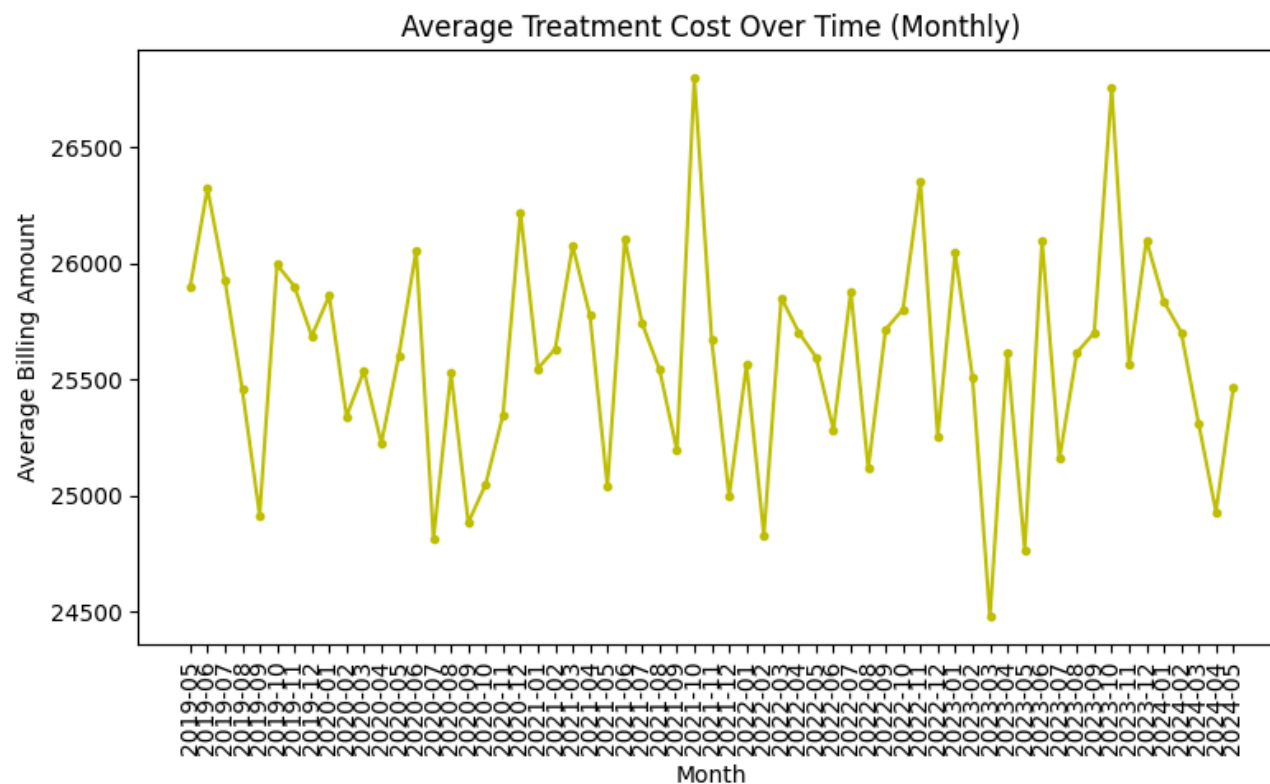
61 rows × 1 columns

dtype: float64

```

1 plt.figure(figsize=(8,5))
2 plt.plot(monthly_avg_count.index.astype(str), monthly_avg_count.values,
3           color = random.choice(colors), marker = 'o', markersize = 3)
4 plt.title('Average Treatment Cost Over Time (Monthly)')
5 plt.xlabel('Month')
6 plt.ylabel('Average Billing Amount')
7 plt.xticks(rotation =90)
8 plt.tight_layout()
9 plt.show()

```



04. Is there a relationship between patient age and treatment cost?

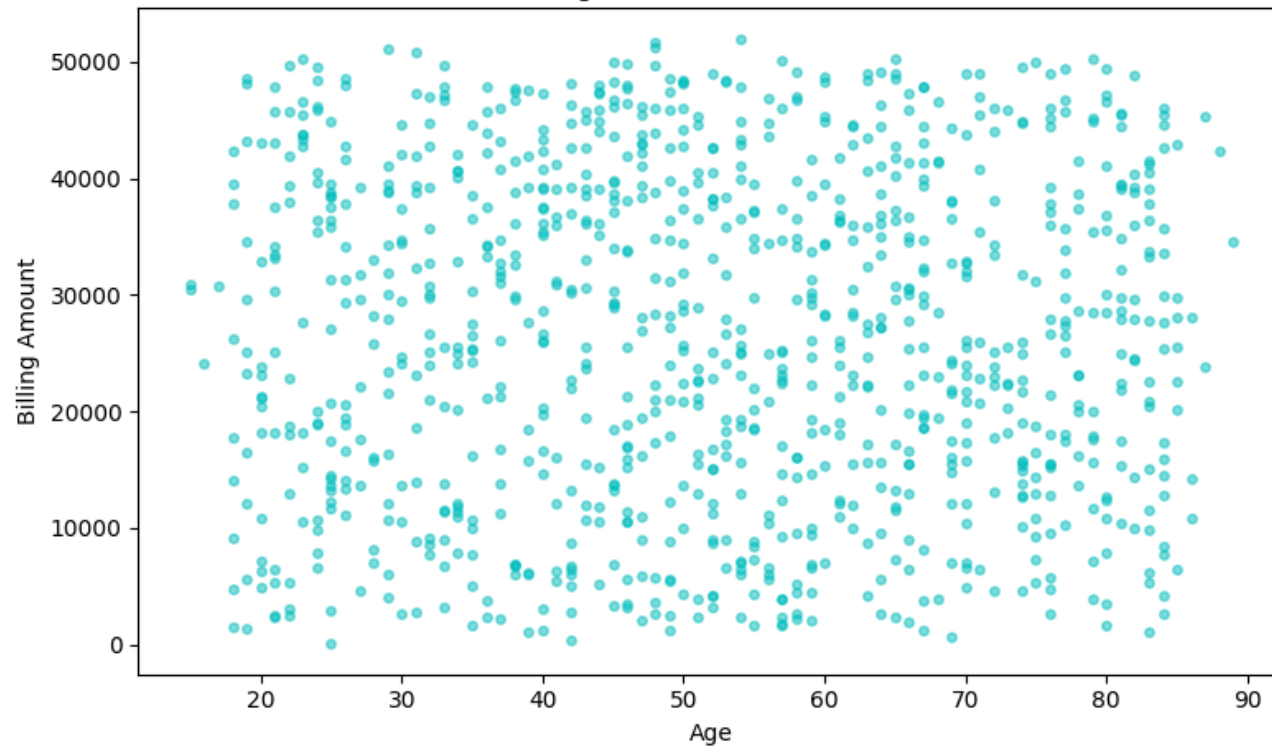
```
1 senior_avg_cost = final_df.loc[final_df['Age']>=60, 'Billing Amount'].mean()
2 senior_avg_cost
```

```
np.float64(25563.587763830074)
```

```
1 sample_df = final_df.sample(1000, random_state = 1)
```

```
1 plt.figure(figsize=(8,5))
2 plt.scatter(sample_df['Age'], sample_df['Billing Amount'], color = random.choice(colors), alpha= 0.5, s = 15)
3 plt.title('Age vs Treatment Cost')
4 plt.xlabel('Age')
5 plt.ylabel('Billing Amount')
6 plt.tight_layout()
7 plt.show()
```

Age vs Treatment Cost



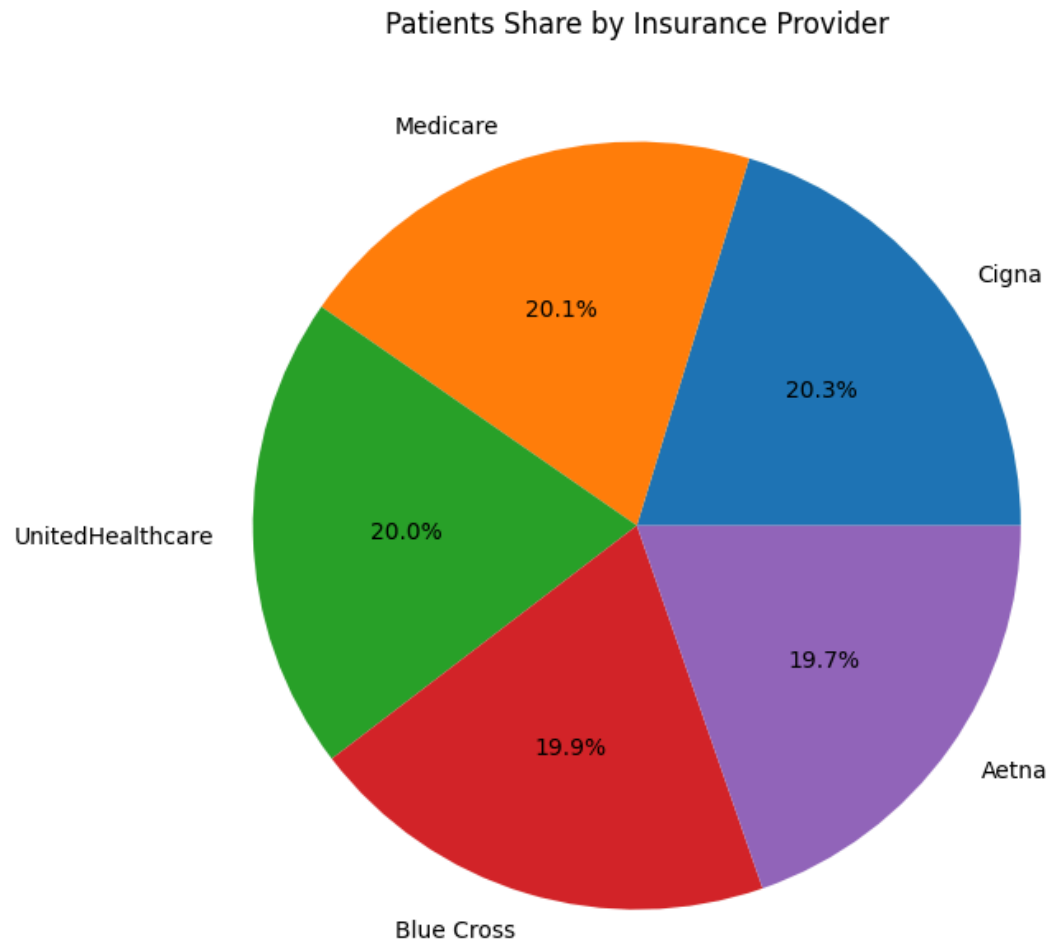
05. Which insurance provider covers the most patients?

```
1 insurance_count = final_df['Insurance Provider'].value_counts()
2 insurance_count
```

Insurance Provider	count
Cigna	11139
Medicare	11039
UnitedHealthcare	11014
Blue Cross	10952
Aetna	10822

dtype: int64

```
1 plt.figure(figsize=(7,7))
2 plt.pie(insurance_count.values, labels =insurance_count.index, autopct = '%1.1f%%')
3 plt.title('Patients Share by Insurance Provider')
4 plt.tight_layout()
5 plt.show()
```



1 Start coding or [generate](#) with AI.

Business Questions

01. Emergency Department Capacity Planning

- The Hospital management is planning staffing for emergency cases. They want to know how many emergency admissions occurred in each month?

```
1 final_df['Admission Type'].value_counts()
```

count	
Admission Type	
Elective	18473
Urgent	18391
Emergency	18102

dtype: int64

```
1 emergency_month = (final_df.loc[final_df['Admission Type'] == 'Emergency'].groupby(final_df['Date of Admission'].dt.month).size())
2 emergency_month
```

0	
Date of Admission	
1	1547
2	1452
3	1528
4	1473
5	1533
6	1483
7	1573
8	1533
9	1500
10	1490
11	1462
12	1528

dtype: int64

- Scenario : Hospital finance team wants to identify patients who have unusually high billing amounts so they can review these cases

```
1 high_cost_patients = final_df.loc[final_df['Billing Amount']> final_df['Billing Amount'].quantile(0.90)]
2 high_cost_patients
```

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
5	Emily Johnson	36	Male	A+	Asthma	2023-12-20	Taylor Newton	Nunez-Humphrey	UnitedHealthcare	48145.110951	389	Urgent	2023-12-24	Ibuprofen	Normal
7	Christina Martinez	20	Female	A+	Cancer	2021-12-28	Suzanne Thomas	Powell Robinson and Valdez,	Cigna	45820.462722	277	Emergency	2022-01-07	Paracetamol	Inconclusive
8	Jasmine Aguilar	82	Male	AB+	Asthma	2020-07-01	Daniel Ferguson	Sons Rich and	Cigna	50119.222792	316	Elective	2020-07-14	Aspirin	Abnormal
30	Thomas Martinez	34	Male	B-	Asthma	2019-08-18	Jacob Huynh	Hart Ltd	Cigna	47909.128810	371	Urgent	2019-09-01	Ibuprofen	Inconclusive
35	David Anderson	25	Female	AB+	Asthma	2021-01-07	Emma Allison	Sons and Cox	Medicare	45453.418546	228	Elective	2021-01-26	Lipitor	Abnormal
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
55446	Richard Moore	63	Female	B+	Hypertension	2021-11-17	Dr. Jacob Aguilar	Blevins Group	Aetna	47649.696638	304	Emergency	2021-12-09	Penicillin	Abnormal
55465	Amanda Stevens	71	Female	O-	Asthma	2019-06-11	Jessica Ramirez	Sims-Henderson	Cigna	45597.476264	164	Urgent	2019-06-22	Ibuprofen	Inconclusive